LAB 2

Containerization & Kubernetes Orchestration

Task 3 — Docker Storage

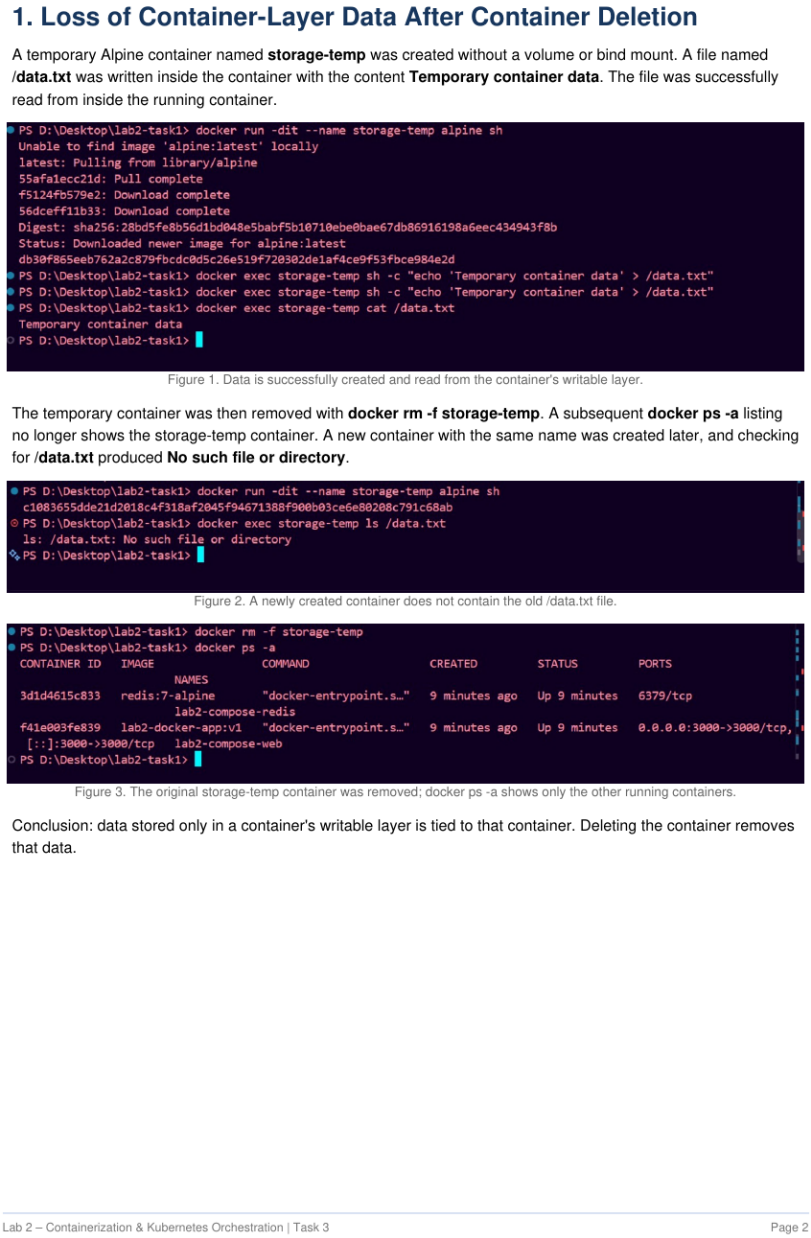
Container-layer storage • Named volume • Bind mount

Student	Kunal Prajapati
Storage methods	Container writable layer, Docker named volume, bind mount
Named volume	lab2-volume
Bind source	.\bind-data

This corrected report reorganizes the supplied evidence so each screenshot appears only with the storage operation it actually demonstrates. The earlier bind-container name-conflict message is omitted from the evidence sequence; the successful bind-mount commands and persistence verification are retained.

1. Loss of Container-Layer Data After Container Deletion

A temporary Alpine container named **storage-temp** was created without a volume or bind mount. A file **/data.txt** was written and read successfully. The container was then removed and a newly created container did not contain the old file.



1. Loss of Container-Layer Data After Container Deletion

A temporary Alpine container named **storage-temp** was created without a volume or bind mount. A file named /**data.txt** was written inside the container with the content **Temporary container data**. The file was successfully read from inside the running container.

```
PS D:\Desktop\lab2-task1> docker run -dit --name storage-temp alpine sh
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
55afa1ecc21d: Pull complete
f5124fb579e2: Download complete
56dceff11b33: Download complete
Digest: sha256:28bd5fe8b56d1bd048e5babf5b10710ebe0bae67db86916198a6eec434943f8b
Status: Downloaded newer image for alpine:latest
db30f865eeb762a2c879fbcdc0d5c26e519f720302de1af4ce9f53fbce984e2d
PS D:\Desktop\lab2-task1> docker exec storage-temp sh -c "echo 'Temporary container data' > /data.txt"
PS D:\Desktop\lab2-task1> docker exec storage-temp sh -c "echo 'Temporary container data' > /data.txt"
PS D:\Desktop\lab2-task1> docker exec storage-temp cat /data.txt
Temporary container data
PS D:\Desktop\lab2-task1>
```

Figure 1. Data is successfully created and read from the container's writable layer.

The temporary container was then removed with **docker rm -f storage-temp**. A subsequent **docker ps -a** listing no longer shows the storage-temp container. A new container with the same name was created later, and checking for /**data.txt** produced **No such file or directory**.

```
PS D:\Desktop\lab2-task1> docker run -dit --name storage-temp alpine sh
c1083655dde21d2018c4f318af2045f94671388f900b03ce6e80208c791c68ab
PS D:\Desktop\lab2-task1> docker exec storage-temp ls /data.txt
ls: /data.txt: No such file or directory
PS D:\Desktop\lab2-task1>
```

Figure 2. A newly created container does not contain the old /data.txt file.

```
PS D:\Desktop\lab2-task1> docker rm -f storage-temp
PS D:\Desktop\lab2-task1> docker ps -a
CONTAINER ID   IMAGE                COMMAND                  CREATED         STATUS         PORTS
                     NAMES
3d1d4615c833   redis:7-alpine       "docker-entrypoint.s…"   9 minutes ago   Up 9 minutes   6379/tcp
                     lab2-compose-redis
f41e003fe839   lab2-docker-app:v1   "docker-entrypoint.s…"   9 minutes ago   Up 9 minutes   0.0.0.0:3000->3000/tcp,
  [::]:3000->3000/tcp   lab2-compose-web
PS D:\Desktop\lab2-task1>
```

Figure 3. The original storage-temp container was removed; docker ps -a shows only the other running containers.

Conclusion: data stored only in a container's writable layer is tied to that container. Deleting the container removes that data.

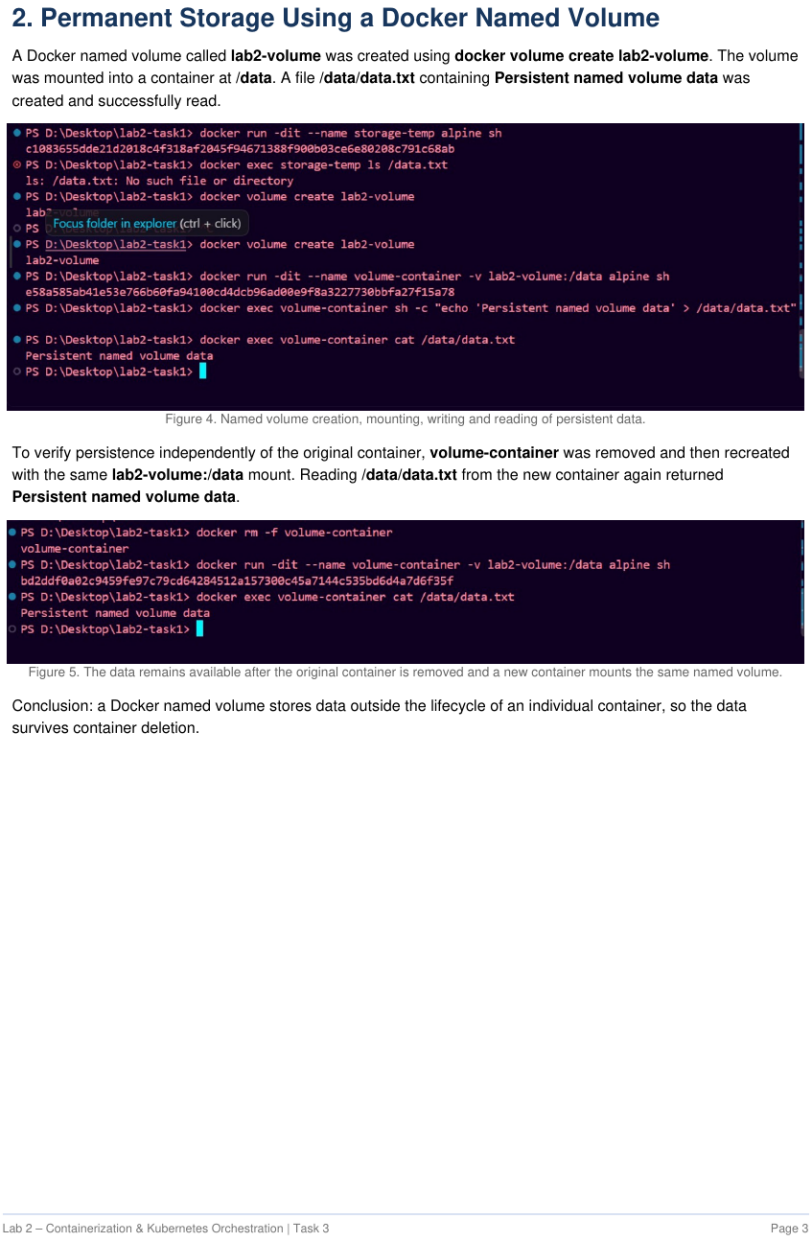
Lab 2 – Containerization & Kubernetes Orchestration | Task 3 Page 2

Figure 1. Original terminal evidence showing creation/read of /data.txt, removal of storage-temp, and verification that a new container has no /data.txt.

Result: Data stored only in a container's writable layer is tied to that container and is lost when the container is deleted.

2. Permanent Storage Using a Docker Named Volume

A Docker named volume called **lab2-volume** was created and mounted at **/data**. The file **/data/data.txt** was written with the content **Persistent named volume data** and read successfully.



2. Permanent Storage Using a Docker Named Volume

A Docker named volume called **lab2-volume** was created using **docker volume create lab2-volume**. The volume was mounted into a container at /**data**. A file /**data**/**data.txt** containing **Persistent named volume data** was created and successfully read.

```
PS D:\Desktop\lab2-task1> docker run -dit --name storage-temp alpine sh
c1083655dde21d2018c4f318af2045f94671388f900b03ce6e80208c791c68ab
PS D:\Desktop\lab2-task1> docker exec storage-temp ls /data.txt
ls: /data.txt: No such file or directory
PS D:\Desktop\lab2-task1> docker volume create lab2-volume
lab
PS
Focus folder in explorer (ctrl + click)
PS D:\Desktop\lab2-task1> docker volume create lab2-volume
lab2-volume
PS D:\Desktop\lab2-task1> docker run -dit --name volume-container -v lab2-volume:/data alpine sh
e58a585ab41e53e766b60fa94100cd4dcb96ad00e9f8a3227730bbfa27f15a78
PS D:\Desktop\lab2-task1> docker exec volume-container sh -c "echo 'Persistent named volume data' > /data/data.txt"

PS D:\Desktop\lab2-task1> docker exec volume-container cat /data/data.txt
Persistent named volume data
PS D:\Desktop\lab2-task1>
```

Figure 4. Named volume creation, mounting, writing and reading of persistent data.

To verify persistence independently of the original container, **volume-container** was removed and then recreated with the same **lab2-volume:/data** mount. Reading /**data**/**data.txt** from the new container again returned **Persistent named volume data**.

```
PS D:\Desktop\lab2-task1> docker rm -f volume-container
volume-container
PS D:\Desktop\lab2-task1> docker run -dit --name volume-container -v lab2-volume:/data alpine sh
bd2ddf0a02c9459fe97c79cd64284512a157300c45a7144c535bd6d4a7d6f35f
PS D:\Desktop\lab2-task1> docker exec volume-container cat /data/data.txt
Persistent named volume data
PS D:\Desktop\lab2-task1>
```

Figure 5. The data remains available after the original container is removed and a new container mounts the same named volume.

Conclusion: a Docker named volume stores data outside the lifecycle of an individual container, so the data survives container deletion.

Figure 2. Original terminal evidence showing named-volume creation, mounting, writing and reading, followed by container recreation using the same volume.

Result: The same data was available after the original container was removed and a new container mounted **lab2-volume:/data**.

3. Permanent Storage Using a Bind Mount

The host directory **.\bind-data** was mounted to **/data**. The successful command wrote **Persistent bind mount data** to **/data/bind.txt**.

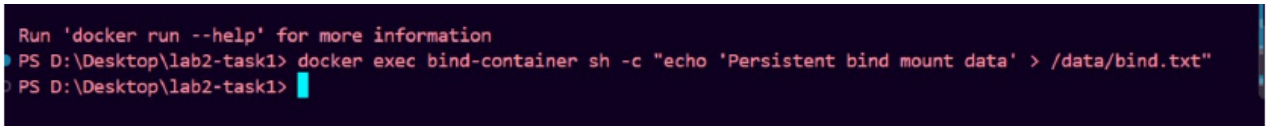


Figure 3. Successful bind-mount command from the original terminal evidence.

The file was then read directly from the Windows host using **Get-Content .\bind-data\bind.txt**.

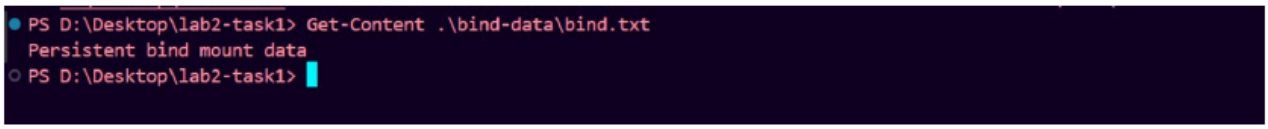


Figure 4. Original host-side verification showing the persistent bind-mount data.

Finally, **bind-container** was removed and the host file was checked again; the same content remained.

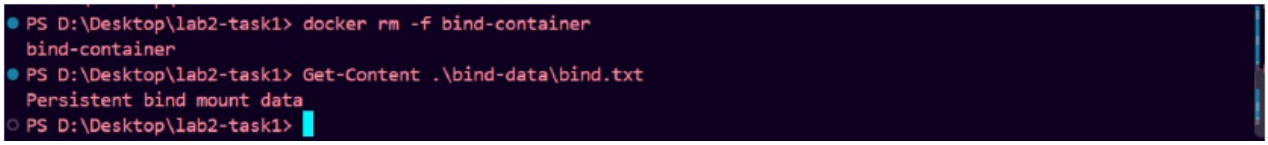


Figure 5. Original terminal evidence showing container removal followed by successful host-file verification.

Result: Bind-mount data remains in the host directory after container deletion.

4. Final Verification

All three storage requirements are demonstrated by the supplied terminal evidence.

Requirement	Demonstration	Result
Container-layer loss	Created /data.txt in storage-temp, removed the container, then verified the file was absent in a new container.	Passed
Named volume	Created lab2-volume, wrote /data/data.txt, recreated the container with the same volume, and read the file again.	Passed
Bind mount	Mounted .\bind-data to /data, wrote bind.txt, verified it on Windows, removed the container, and verified it again.	Passed

Key distinction

Container layer: temporary and lost with the container. **Named volume:** managed by Docker and persists independently of a container. **Bind mount:** data lives directly in the host directory and remains after container deletion.

Submission note: Use this PDF as the Task 3 report and keep the Docker commands/project files in the GitHub repository as supporting source material.